ARCHITECTURE DATA & IA · BLOC 2 · OCTOBRE 2026

Payer en 100 ms, sans jamais perdre un paiement tout en évitant la fraude.

Étude de cas Stripe · Ronan GUILLOU · Certification RNCP41993

01 — LE BESOIN Livrable 0

Les exigences de la plateforme.

100 ms

pour l'acceptation du paiement et la décision de fraude

3

systèmes de données : OLTP, OLAP et NoSQL pour traiter des millions de transactions par jour, pics saisonniers compris

IA

détection de fraude et analyses prédictives pour servir le décisionnel

Conformité

Réglementations : RGPD, CCPA et PCI-DSS · sécurité : chiffrement, accès par rôle, audit

STOCKAGE DES DONNÉES Livrables 1 · 2 · 3

3 systèmes : un stockage adapté à chaque donnée.

Aurora PostgreSQL

OLTP : transactions ACID, Multi-AZ

Lakehouse Databricks

OLAP en étoile & Bronze, Silver, Gold sur S3

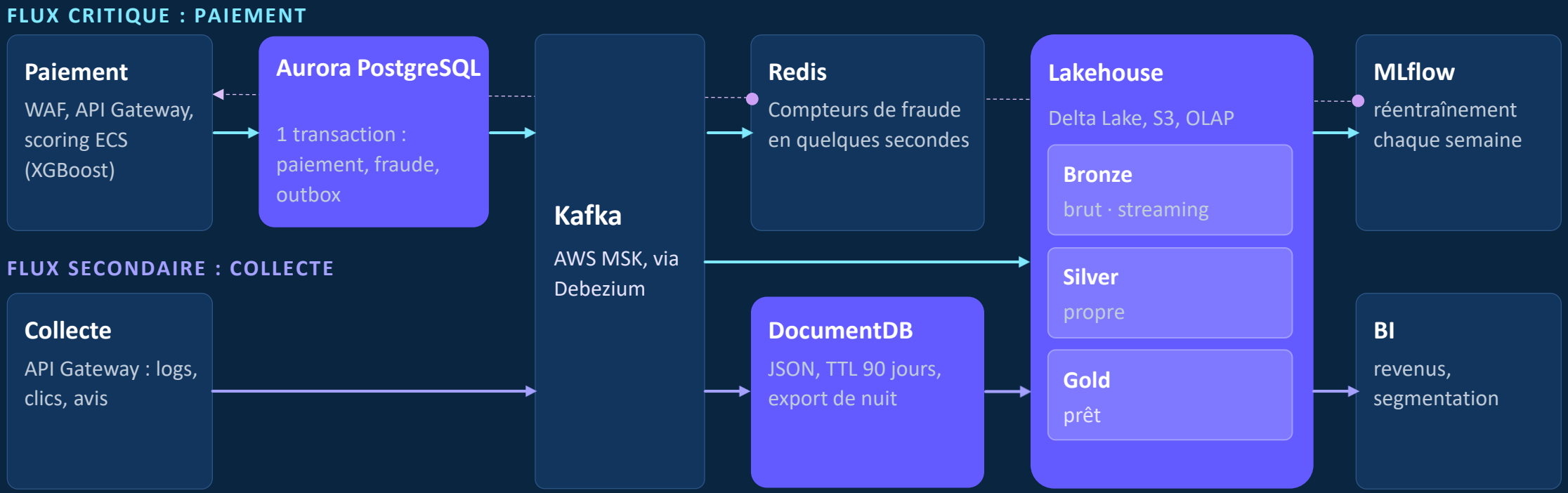
DocumentDB

NoSQL : logs, clics, avis (JSON)

02 — ARCHITECTURE

Livrables 4 · 7

Deux flux d'entrée vers le lakehouse.



Un flux unique : source → Kafka → Bronze → Silver → Gold, sans perte ni doublon.

2 consommateurs de données : détection de fraude (ML) & analyses décisionnelles (BI)

03 — VÉLOCITÉ

Livrables 1 · 3 · 5

Parcours critique de paiement.

01 Scoring en 100 ms

Le modèle XGBoost sur ECS Fargate évalue le risque de fraude avant l'acceptation du paiement.

02 Variables chaudes en cache

Redis, mis à jour en quelques secondes et lu en moins d'une milliseconde, sans interroger les bases.

03 Un chemin court et protégé

Transactions partitionnées par mois, collecte séparée du paiement, temps réel réservé au critique.

REPÈRES

100 ms

pour décider de la fraude, avant l'acceptation du paiement

< 1 ms

pour lire les compteurs de fraude dans Redis

04 — RÉSILIENCE

Livrables 1 · 4 · 7

Résilience : aucun paiement perdu en cas de panne.

01 Infrastructure hautement disponible

Aurora Multi-AZ avec bascule automatique, ECS Fargate, scaling horizontal (ECS) et vertical (Aurora).

02 Atomicité et idempotence

Paiement et événement en une transaction, Debezium publie dans Kafka. Si Kafka tombe, les événements attendent et un rejeu ne crée aucun doublon.

03 Historique et reprise

Bronze conserve tout tel que reçu et sert au rejeu. Les lignes invalides vont en quarantaine ou en DLQ, alerte CloudWatch signale le taux de rejet. Sauvegardes Aurora et Delta (30 jours max), RESTORE Delta, Redis reconstruit depuis les sources.

REPÈRES

99,99 %

de disponibilité. Aucun paiement perdu, même si Kafka est indisponible : l'outbox conserve les événements.

3

systèmes de données synchronisés, sans perte ni doublon.

05 — SÉCURITÉ ET CONFORMITÉ

Livrable 6

Sécurité et conformité : RGPD, CCPA, PCI-DSS.

01 Protéger les données

Chiffrement KMS et TLS, secrets dans Secrets Manager, tokenisation des cartes, pseudonymisation.

02 Contrôler et tracer

Accès par rôle (IAM, Unity Catalog), moindre privilège, CloudTrail, contrôles quotidiens, tables d'audit et CloudWatch.

03 Respecter RGPD, CCPA, PCI-DSS

Accès et demandes d'effacement (paiements conservés, personne anonymisée), classification des données, données UE hébergées en UE, aucun numéro de carte stocké.

REPÈRES

3

réglementations couvertes : RGPD, CCPA et PCI-DSS

20

briques techniques, chacune protégée par défaut (chiffrement, droits par rôle)

06 — INTÉGRATION IA

Livrable 5

Trois cas d'usage, une plateforme.

INSTANTANÉE

Fraude

- XGBoost sur ECS Fargate CPU
- Variables chaudes lues dans Redis
- Cas suspects envoyés à l'équipe fraude

QUOTIDIENNE

Personnalisation

- Filtrage collaboratif sur Databricks
- Résultat servi via Redis
- Produits populaires pour les nouveaux clients, produits fréquemment achetés…

QUOTIDIENNE

Prévisions

- Séries temporelles sur Databricks
- Résultats rendus à la BI
- Pilotage de la direction financière

Cycle de vie : MLflow enregistre un challenger, le Model Owner valide avant mise en production, retour arrière possible. Suivi : dérive des variables, faux refus et temps de réponse sont surveillés. Si un seuil est dépassé, une alerte part et le Model Owner décide : réentraînement ou retour à la version précédente. Chaque décision garde son score, ses motifs et la version du modèle.

07 — SYNTHÈSE

Livrable 1 à 8

Chaque attente du business case a sa réponse.

100 ms

→ slide 5

Attente : décider de la fraude avant l'acceptation

✓ Scoring XGBoost en 100 ms, variables Redis lues en moins d'1 ms

3

→ slides 3 et 4

Attente : OLTP, OLAP et NoSQL, des millions de transactions par jour

✓ Aurora, lakehouse et DocumentDB reliés par Kafka et Debezium, sans perte ni doublon

IA

→ slide 8

Attente : détection de fraude et analyses prédictives

✓ Fraude, personnalisation et prévisions sur une même plateforme MLflow

Conformité

→ slide 7

Attente : RGPD, CCPA, PCI-DSS et sécurité

✓ Chiffrement, accès par rôle, audit, aucun numéro de carte stocké

Millions de transactions, pics saisonniers compris : Aurora Multi-AZ avec bascule automatique, scaling horizontal (ECS) et vertical (Aurora), transactions partitionnées par mois (slides 5 et 6).

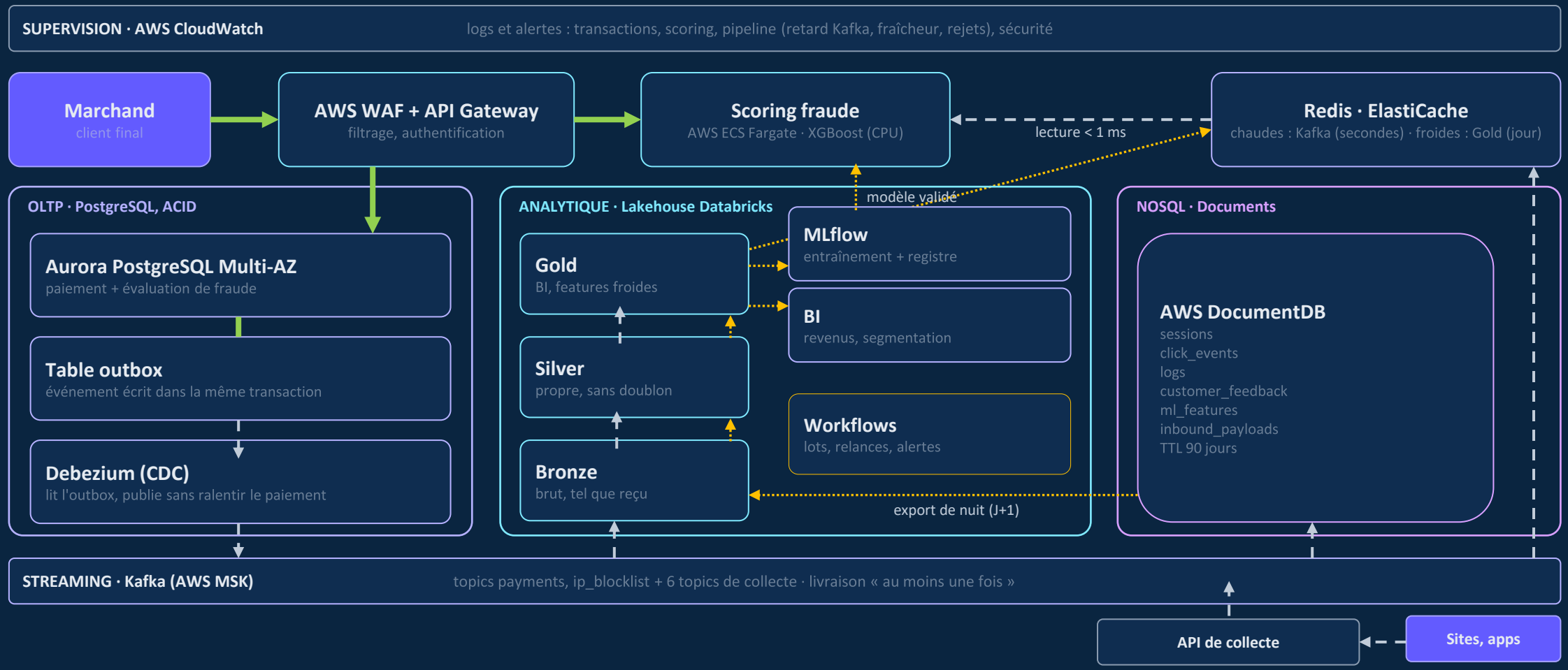
ANNEXE 1

Les choix techniques et leur justification.

DOMAINE	CHOIX RETENU	POURQUOI
Stockage	Aurora PostgreSQL Multi-AZ (OLTP), lakehouse Delta Lake sur S3 (OLAP), DocumentDB (NoSQL)	Un stockage par nature de donnée : ACID pour les paiements, étoile pour l'analyse, JSON pour les formes variables.
Intégrité	Une transaction : paiement, fraude, outbox. Debezium (CDC) publie dans Kafka (MSK)	Aucun événement perdu ni fantôme. Kafka livre « au moins une fois » : consommateurs idempotents (MERGE, upsert, idempotency_key).
Performance	Transactions partitionnées par mois, insertion seule ; Redis (8 compteurs avec TTL) ; collecte séparée du paiement	Décision en moins de 100 ms sans interroger les bases, et sans verrou sur le chemin critique.
Fraîcheur	Temps réel réservé au critique ; micro-batch Bronze vers Silver ; Gold à l'heure ou au jour ; exports de nuit	Chaque usage reçoit la fraîcheur qu'il exige, pas plus : moins de calcul, donc moins de coût et d'énergie.
Qualité et rejeu	Médaillon : Bronze en ajout seul, Silver par MERGE, quarantaine, DLQ, Delta RESTORE	Tout se recalcule depuis Bronze ; aucune ligne rejetée n'est perdue, son motif est conservé.
IA	XGBoost sur ECS Fargate (conteneur CPU) ; entraînement Databricks, registre MLflow, validation du Model Owner	Modèle léger et rapide, entraîné hors du chemin de paiement ; retour arrière possible.
Sécurité	KMS (une clé par domaine, rotation annuelle), TLS, Secrets Manager, IAM, Unity Catalog, tokenisation, zones réseau	Une clé compromise n'expose qu'un domaine ; aucun numéro de carte stocké ; test et production séparés.
Exploitation	Databricks Workflows (dépendances, relances), CloudWatch (retard Kafka, fraîcheur, rejets, sécurité)	Services managés, relances automatiques et alertes : peu d'opérations manuelles.

ANNEXE 2

Architecture complète : les briques et leurs flux.



LÉGENDE — chemin critique < 100 ms — — — quelques secondes ····· par lots (heure, jour)

ANNEXE 3

Les flux retenus, leur chemin et leur fraîcheur.

FLUX	CHEMIN	MODE ET FRAÎCHEUR
Paiement	Marchand, WAF, API Gateway, scoring ECS, Aurora (une seule transaction)	Synchrone, moins de 100 ms
Variables chaudes	Redis vers le scoring, sans interroger les bases	Lecture en moins d'1 ms
Événements de paiement	Aurora (outbox), Debezium, Kafka (topics payments, ip_blocklist)	Continu, quelques secondes
Compteurs de fraude	Kafka vers Redis (compteurs C1 à C8, liste de blocage IP)	Continu, quelques secondes
Paiements vers Bronze	Kafka vers Bronze (Delta, ajout seul, métadonnées Kafka)	Continu, quelques minutes
Collecte	Sites et apps, API de collecte, Kafka, DocumentDB (6 collections)	Continu, quelques secondes
Export du NoSQL	DocumentDB vers Bronze (documents nouveaux seulement)	Chaque nuit (J+1)
Données de référence	Aurora vers Bronze (marchand, client, produit, pays, devise, catégorie)	Chaque nuit
Transformation	Bronze, Silver (MERGE, quarantaine), Gold (dimensions avant les faits)	Silver : minutes · Gold : heure ou jour
Variables froides	Gold vers Redis	Chaque jour
Entraînement, déploiement	Gold, MLflow, validation du Model Owner, redéploiement sur ECS	Chaque semaine
BI et prévisions	Gold vers BI ; résultats servis via Redis ou rendus à la BI	Lecture directe · chaque jour